

Web Application Development

Lecture 14: MongoDB & Mongoose

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Create a **MongoDB Atlas** free cluster and obtain a connection string
- Connect an Express app to Atlas with `mongoose.connect()`
- Define a **Schema** with types, required, default, and trim validators
- Create a **Model** and understand automatic collection naming
- Perform all CRUD operations using Mongoose: **find**, **findById**, **create**, **findByIdAndUpdate**, **findByIdAndDelete**
- Replace in-memory arrays in routes with **real database calls**
- Add a **bcrypt pre-save hook** to the User model
- Enforce **input sanitisation** and **ownership checks** in routes

Agenda

- 1 MongoDB Atlas Setup
- 2 `mongoose.connect()`
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations
- 5 User Model with bcrypt pre-save hook
- 6 Ownership, Sanitisation & Wiring
- 7 Recap

Outline

- 1 MongoDB Atlas Setup
- 2 `mongoose.connect()`
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations
- 5 User Model with bcrypt pre-save hook
- 6 Ownership, Sanitisation & Wiring
- 7 Recap

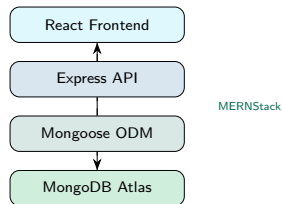
What is MongoDB Atlas?

MongoDB Atlas – Cloud Database

MongoDB Atlas is a **fully managed cloud database** service. It handles deploying, managing, scaling, and securing MongoDB without any local installation. A **free M0 cluster** provides 512 MB of storage, enough for learning and small projects.

SQL vs MongoDB terminology:

SQL	MongoDB	Mongoose
Database	Database	Database
Table	Collection	Model
Row	Document	Document object
Column	Field	Schema field
Primary key	_id	Auto-generated



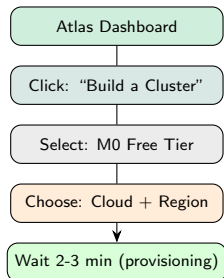
W3Schools – MongoDB + Express

MongoDB stores data as **flexible JSON-like documents**, letting you adjust data structures with minimal overhead. Together with Express.js it lets you build dynamic, scalable, and efficient backends.

Atlas Setup – Step 1 & 2: Account and Cluster

Step 1 – Create a free account:

- 1 Go to **mongodb.com/cloud/atlas**
- 2 Click “Try Free” – register with email
- 3 Choose the **FREE M0 plan**



Step 2 – Create a database user:

- 1 In left menu: click “Database Access”
- 2 Click “Add New Database User”
- 3 Choose: **Password authentication**
- 4 Set username: e.g. swe381user
- 5 Set a **strong password** (save it!)
- 6 Role: **Atlas admin** (for learning)
- 7 Click **Add User**

Separate from Atlas login

The database user is **different** from your Atlas account login. This username/password goes into your MONGO_URI connection string.

Atlas Setup – Step 3: IP Whitelist and Connection String

Step 3 – Whitelist your IP:

- 1 Left menu: click “Network Access”
- 2 Click “Add IP Address”
- 3 For development: click “Allow Access from Anywhere” (0.0.0.0/0)
- 4 Click Confirm

Restrict IPs in production

0.0.0.0/0 allows any IP – fine for class. In production, restrict to your server's IP for better security.

Step 4 – Get your connection string:

- 1 Clusters page: click “Connect”
- 2 Choose: “Connect your application”
- 3 Select: Node.js driver
- 4 Copy the connection string

Step 5 – Paste in .env:

```
# .env
PORT=3000
MONGO_URI=mongodb+srv://swe381user:
<PASSWORD>@cluster0.abc12.mongodb.net/
swe381db?retryWrites=true&w=majority
JWT_SECRET=swe381-super-secret-key
FRONTEND_URL=http://localhost:5173
```

Replace <PASSWORD> and swe381db with your actual values.

Outline

- 1 MongoDB Atlas Setup
- 2 `mongoose.connect()`
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations
- 5 User Model with bcrypt pre-save hook
- 6 Ownership, Sanitisation & Wiring
- 7 Recap

Connecting to MongoDB Atlas

```
$ npm install mongoose
```

config/db.js – connection module:

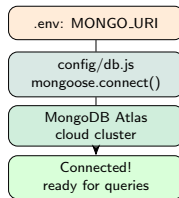
```
1 const mongoose = require('mongoose');
2
3 const connectDB = async () => {
4   try {
5     await mongoose.connect(
6       process.env.MONGO_URI
7     );
8     console.log('MongoDB Atlas connected');
9   } catch (err) {
10    console.error('DB connection failed:', err);
11    process.exit(1); // stop the server
12  }
13 };
14
15 module.exports = connectDB;
```

index.js – call before listen():

```
1 require('dotenv').config();
2 const express = require('express');
3 const connectDB = require('./config/db');
4 const app = express();
5
6 // 1. Connect to DB first
7 connectDB();
8
9 app.use(express.json());
```

Terminal on npm run dev

MongoDB Atlas connected
Server running on port 3000



! process.exit(1)

If the DB connection fails, the server **must** exit. Without a DB, routes that need data will crash on every request.

Outline

- 1 MongoDB Atlas Setup
- 2 mongoose.connect()
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations
- 5 User Model with bcrypt pre-save hook
- 6 Ownership, Sanitisation & Wiring
- 7 Recap

Mongoose Schema – Defining Document Structure

W3Schools – Express.js with MongoDB

You use **Mongoose schemas and models** to define the shape of your data. A schema defines the structure of your collection documents. The first argument you pass to the model is the **singular form of your collection name** – Mongoose automatically converts it to plural lowercase for the collection.

```
1 // models/Course.js
2 const mongoose = require('mongoose');
3
4 const courseSchema = new mongoose.Schema({
5   // --- Field options ---
6   title: {
7     type: String, // data type
8     required: [true, 'Course title is required'],
9     trim: true, // strip whitespace
10    minlength:[3, 'Title must be >= 3 chars']
11  },
12  dept: {
13    type: String,
14    required: [true, 'Department is required'],
15    trim: true,
16    uppercase:true // store as uppercase
17  },
```

```
1   credits: {
2     type: Number,
3     default: 3, // used when not provided
4     min: [1, 'Credits must be >= 1'],
5     max: [6, 'Credits cannot exceed 6']
6   },
7   isActive: {
8     type: Boolean,
9     default: true
10  }
11 }, { timestamps: true }); // adds createdAt/updatedAt
12
13 module.exports = mongoose.model(
14   'Course', courseSchema);
15 // 'Course' --> collection: 'courses'
```

Schema Types Reference

Type	Example value	Use case
String	"SWE 381"	names, titles, emails
Number	3	credits, age, price
Boolean	true	flags, toggles
Date	new Date()	timestamps
ObjectId	_id ref	foreign key / ref
Array	["a","b"]	tags, lists

Common field options:

Option	Purpose
required	Field must be present
default	Value when not provided
trim	Strip whitespace
uppercase	Auto-uppercase
lowercase	Auto-lowercase
unique	No duplicate values
min/max	Range for Numbers
minlength	Minimum string length

```
1 // Compact schema syntax (shorthand)
2 const courseSchema = new mongoose.Schema({
3   title: String,           // just the type
4   credits: Number,
5   isActive: Boolean
6 });
7
8 // Full syntax (with validation)
9 const courseSchema = new mongoose.Schema({
10   title: {
11     type: String,
12     required: true,
13     trim: true
14   },
15   credits: {
16     type: Number,
17     default: 3
18   }
19 });
```



Use full syntax in production

The shorthand is fine for prototyping. Use the full object syntax when you need **required**, **trim**, **default**, or any validation on a field.

Model Creation and Collection Naming

W3Schools – Mongoose Models

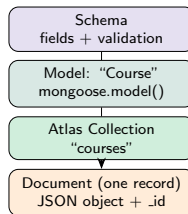
Models take your schema and apply it to each document in its collection. The **first argument** you pass to `mongoose.model()` is the **singular form** of your collection name. Mongoose automatically changes this to the **plural form** and transforms it to lowercase.

```
// 1. Define schema
const courseSchema = new mongoose.Schema({
  title: { type: String, required: true },
  dept: { type: String, required: true },
  credits: { type: Number, default: 3 }
}, { timestamps: true });

// 2. Create model (singular name)
const Course = mongoose.model(
  'Course', // <-- singular
  courseSchema
);

// Mongoose creates collection: 'courses'
// 'Course' --> 'courses'
// 'Student' --> 'students'
// 'Category' --> 'categories'

// 3. Export for use in routes/controllers
module.exports = Course;
```



Document in Atlas

```
{
  _id: ObjectId("64..."),
  title: "Web App Dev",
  dept: "SWE",
```

Outline

- 1 MongoDB Atlas Setup
- 2 mongoose.connect()
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations**
- 5 User Model with bcrypt pre-save hook
- 6 Ownership, Sanitisation & Wiring
- 7 Recap

find() and findById() – Reading Data

```
1 const Course = require('../models/Course');
2
3 // GET /api/courses -- find ALL
4 router.get('/', async (req, res, next) => {
5   try {
6     const courses = await Course.find();
7     res.status(200).json(courses);
8   } catch (err) {
9     next(err);
10  }
11 });
12
13 // GET /api/courses?dept=SWE -- filter
14 router.get('/', async (req, res, next) => {
15   try {
16     const filter = {};
17     if (req.query.dept) {
18       filter.dept = req.query.dept;
19     }
20     const courses = await Course.find(filter);
21     res.status(200).json(courses);
22   } catch (err) {
23     next(err);
24   }
25 });
26
27 // GET /api/courses/:id -- find ONE
28 router.get('/:id', async (req, res, next) => {
29   try {
30     const course =
31       await Course.findById(req.params.id);
32     if (!course) {
33       return res.status(404).json({
34         error: 'Course not found'
35       });
36     }
37     res.status(200).json(course);
38   } catch (err) {
39     next(err);
40   }
41 });
```

GET /api/courses

```
[
  { "id": "64a...", "title": "Web App Dev" },
  { "id": "64b...", "title": "Algorithms" }
]
```

GET /api/courses?dept

```
[
  { "title": "Web App Dev", "dept": "SWE" },
  { "title": "SW Design", "dept": "SWE" }
]
```

GET /api/courses/64a...

```
{
  "_id": "64a...",
  "title": "Web App Dev",
  "dept": "SWE", "credits": 3
}
```

create() – Inserting a Document

```
1 const Course = require('../models/Course');
2
3 // POST /api/courses -- create new
4 router.post('/', protect, async (req, res, next) => {
5   try {
6     let { title, dept, credits } = req.body;
7
8     // Input sanitisation: reject whitespace-only
9     if (!title || !title.trim()) {
10       return res.status(400).json({
11         error: 'Title cannot be empty or whitespace'
12       });
13     }
14     if (!dept || !dept.trim()) {
15       return res.status(400).json({
16         error: 'Dept cannot be empty or whitespace'
17       });
18     }
19
20     // Store who created this document
21     const course = await Course.create({
22       title: title.trim(),
23       dept: dept.trim().toUpperCase(),
24       credits: credits || 3,
25       user: req.user.id // from protect middleware
26     });
27
28     res.status(201).json(course);
29   } catch (err) {
30     next(err); // Mongoose validation errors go here
31   }
32 });
```

POST body:

```
{
  "title": "Cloud Computing",
  "dept": "cs",
  "credits": 3
}
```

201 Created

```
{
  "_id": "64c...",
  "title": "Cloud Computing",
  "dept": "CS",
  "credits": 3,
  "user": "64u...",
  "createdAt": "2025-01-20T...",
  "__v": 0
}
```

💡 Course.create() vs new + save()

Course.create(data) is shorthand for new Course(data).save(). Both trigger Mongoose validation and the pre('save') hook.

findByIdAndUpdate() – Updating a Document

```
1 // PUT /api/courses/:id
2 router.put('/:id', protect, async (req, res, next) => {
3   try {
4     // 1. Find first (for ownership check)
5     const course =
6       await Course.findById(req.params.id);
7
8     if (!course) {
9       return res.status(404).json({
10         error: 'Course not found'
11       });
12     }
13
14     // 2. Ownership check
15     if (course.user.toString() !== req.user.id) {
16       return res.status(403).json({
17         error: 'Not authorised to update this course'
18       });
19     }
20
21     // 3. Update document
22     const updated = await Course.findByIdAndUpdate(
23       req.params.id,
24       req.body,
25       {
26         new: true, // return the UPDATED doc
27         runValidators: true // run schema validation
28       }
29     );
30
31     res.status(200).json(updated);
32   } catch (err) {
33     next(err);
34   }
35 });
```

PUT body:

```
{
  "title": "Cloud Computing v2",
  "credits": 4
}
```

200 OK – updated doc

```
{
  "_id": "64c...",
  "title": "Cloud Computing v2",
  "dept": "CS",
  "credits": 4,
  "updatedAt": "2025-01-20T..."
}
```

Option	Meaning
new: true	return updated doc
runValidators: true	check schema rules

findByIdAndDelete() – Removing a Document

```
1 // DELETE /api/courses/:id
2 router.delete('/:id', protect,
3   async (req, res, next) => {
4     try {
5       // 1. Find the document
6       const course =
7         await Course.findById(req.params.id);
8
9       if (!course) {
10        return res.status(404).json({
11          error: 'Course not found'
12        });
13      }
14
15      // 2. Ownership check (403 not 404!)
16      if (course.user.toString() !== req.user.id) {
17        return res.status(403).json({
18          error: 'Not authorised to delete'
19        });
20      }
21
22      // 3. Delete the document
23      await Course.findByIdAndDelete(req.params.id);
24
25      res.status(200).json({
26        message: 'Course deleted successfully'
27      });
28
29    } catch (err) {
30      next(err);
31    }
32  }
33 );
```

DELETE /api/courses/64c – 200

```
{
  "message": "Course deleted successfully"
}
```

Wrong owner – 403

```
{
  "error": "Not authorised to delete"
}
```

⚠ 403 vs 404

404 Not Found: the document doesn't exist at all.

403 Forbidden: the document exists but this user has no permission.

Never return 403 before confirming the document exists – otherwise you leak the existence of other users' data.

Mongoose CRUD – Quick Reference

Operation	Mongoose method	HTTP	Status
Read all	<code>Course.find()</code>	GET	200
Read one	<code>Course.findById(id)</code>	GET	200 / 404
Create	<code>Course.create(data)</code>	POST	201
Update	<code>Course.findByIdAndUpdate(id, data, opts)</code>	PUT	200 / 404
Delete	<code>Course.findByIdAndDelete(id)</code>	DELETE	200 / 404

```
1 // Useful query modifiers
2 // Filter + sort + limit
3 await Course.find({ dept: 'SWE' })
4   .sort({ createdAt: -1 }) // newest first
5   .limit(10);
6
7 // Select specific fields only
8 await Course.findById(id)
9   .select('title dept credits');
10
11 // Exclude a field
12 await Course.findById(id)
13   .select('-__v -user');
```

```
1 // Count documents
2 const count = await Course.countDocuments({
3   dept: 'SWE'
4 });
5
6 // Check if document exists
7 const exists = await Course.exists({
8   title: 'Web App Dev'
9 });
10
11 // findByIdAndUpdate options
12 { new: true, runValidators: true }
```

Outline

- 1 MongoDB Atlas Setup
- 2 mongoose.connect()
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations
- 5 User Model with bcrypt pre-save hook**
- 6 Ownership, Sanitisation & Wiring
- 7 Recap

User Model with bcrypt pre-save Hook

Mongoose Middleware – pre-save hook

A **pre-save hook** is middleware that runs **before** a document is saved. A common use case is to **automatically hash the password** before storing it, so the controller never has to call `bcrypt.hash()` manually.

```
// models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true,
    lowercase: true, trim: true },
  password: { type: String, required: true, minlength: 8,
    select: false }, // excluded from queries by default
  role: { type: String, enum: ['student', 'staff', 'admin'],
    default: 'student' }
}, { timestamps: true });

// Pre-save hook: hash password automatically before save
userSchema.pre('save', async function(next) {
  // Only hash if password was actually modified
  if (!this.isModified('password')) return next();

  this.password = await bcrypt.hash(this.password, 10);
  next();
});

module.exports = mongoose.model('User', userSchema);
```

Using the User Model in Auth Routes

routes/authRoutes.js (simplified):

```
1 const User = require('../models/User');
2 const bcrypt = require('bcrypt');
3 const jwt = require('jsonwebtoken');
4
5 // POST /api/auth/register
6 router.post('/register', async (req, res, next) => {
7   try {
8     const { name, email, password } = req.body;
9
10    // pre-save hook hashes password automatically!
11    const user = await User.create({
12      name,
13      email,
14      password // plain text -- hook hashes it
15    });
16
17    res.status(201).json({
18      message: 'Registered',
19      userId: user._id
20    });
21  } catch (err) {
22    // Mongoose duplicate key error code
23    if (err.code === 11000) {
24      return res.status(400).json({
25        error: 'Email already registered'
26      });
27    }
28    next(err);
29  }
30 });
```

```
1 // POST /api/auth/login
2 router.post('/login', async (req, res, next) => {
3   try {
4     const { email, password } = req.body;
5
6     // .select('+password') re-includes
7     // the excluded field for comparison
8     const user = await User.findOne({ email })
9       .select('+password');
10
11     if (!user) {
12       return res.status(401).json({
13         error: 'Invalid email or password'
14       });
15     }
16
17     const match = await bcrypt.compare(
18       password, user.password
19     );
20     if (!match) {
21       return res.status(401).json({
22         error: 'Invalid email or password'
23       });
24     }
25
26     const token = jwt.sign(
27       { id: user._id, role: user.role },
28       process.env.JWT_SECRET,
29       { expiresIn: '7d' }
30     );
31     res.json({ token });
32   } catch (err) { next(err); }
33 });
```

Outline

- 1 MongoDB Atlas Setup
- 2 `mongoose.connect()`
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations
- 5 User Model with bcrypt pre-save hook
- 6 Ownership, Sanitisation & Wiring**
- 7 Recap

Ownership Check Pattern

Ownership Check

Store **who created** a document at creation time. On update/delete, **compare the stored user ID** with the requesting user's ID. Return **403 Forbidden** (not 404) if they don't match.

Step 1: Add user field to schema:

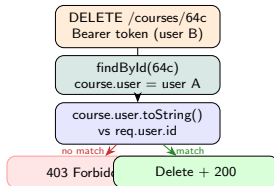
```
1 // models/Course.js (add user field)
2 user: {
3   type:      mongoose.Schema.Types.ObjectId,
4   ref:       'User',
5   required:  true
6 }
```

Step 2: Store user on create:

```
1 // POST /api/courses
2 const course = await Course.create({
3   title: title.trim(),
4   dept:  dept.trim(),
5   credits,
6   user:  req.user.id // from protect middleware
7 });
```

Step 3: Check ownership on update/delete:

```
1 const course = await Course.findById(id);
2
3 // course.user is ObjectId, req.user.id is string
4 if (course.user.toString() !== req.user.id) {
5   return res.status(403).json({
```



💡 .toString() is required

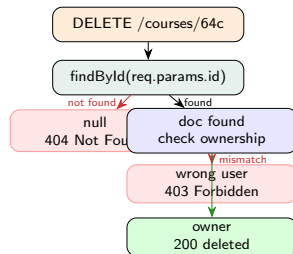
Mongoose stores the user field as an **ObjectId** object. req.user.id is a string. Use **.toString()** to compare them correctly.

Input Sanitisation and 403 vs 404

Input sanitisation best practices:

```
1 router.post('/', protect, async (req, res, next) => {
2   let { title, dept, credits } = req.body;
3
4   // 1. Reject missing/null values
5   if (!title || !dept) {
6     return res.status(400).json({
7       error: 'title and dept are required'
8     });
9   }
10
11  // 2. Reject whitespace-only strings
12  if (!title.trim() || !dept.trim()) {
13    return res.status(400).json({
14      error: 'Fields cannot be whitespace only'
15    });
16  }
17
18  // 3. Normalise before saving
19  title = title.trim();
20  dept = dept.trim().toUpperCase();
21  credits = Number(credits) || 3;
22
23  const course = await Course.create({
24    title, dept, credits,
25    user: req.user.id
26  });
27
28  res.status(201).json(course);
29 };
```

403 Forbidden vs 404 Not Found:



Replacing In-Memory Arrays – Full Route Comparison

Before: in-memory array (Lecture 11):

```
1 // routes/courseRoutes.js
2 let courses = [
3   { id: 1, title: "Web App Dev", dept: "SWE" },
4 ];
5 let nextId = 2;
6
7 router.get('/', (req, res) => {
8   res.json(courses);
9 });
10
11 router.post('/', (req, res) => {
12   const course = {
13     id: nextId++,
14     ...req.body
15   };
16   courses.push(course);
17   res.status(201).json(course);
18 });
19
20 router.delete('/:id', (req, res) => {
21   courses = courses.filter(
22     c => c.id !== parseInt(req.params.id)
23   );
24   res.json({ message: 'Deleted' });
25 });
```

After: real MongoDB (Lecture 14):

```
1 // routes/courseRoutes.js
2 const Course = require('../models/Course');
3
4 router.get('/', async (req, res, next) => {
5   try {
6     const courses = await Course.find();
7     res.status(200).json(courses);
8   } catch (err) { next(err); }
9 });
10
11 router.post('/', protect, async (req, res, next) => {
12   try {
13     const course = await Course.create({
14       ...req.body,
15       user: req.user.id
16     });
17     res.status(201).json(course);
18   } catch (err) { next(err); }
19 });
20
21 router.delete('/:id', protect,
22   async (req, res, next) => {
23     try {
24       const c = await Course.findById(req.params.id);
25       if (!c) return res.status(404).json({ error: 'Not found'
26         });
27       if (c.user.toString() !== req.user.id)
28         return res.status(403).json({ error: 'Forbidden' });
29       await Course.findByIdAndDelete(req.params.id);
30       res.json({ message: 'Deleted' });
31     } catch (err) { next(err); }
```

Testing with Postman – Real MongoDB Data

End-to-end Postman workflow:

- 1 **Start server:** `npm run dev` – confirm MongoDB Atlas connected
- 2 **Register:** `POST /api/auth/register` – creates a user in Atlas users collection
- 3 **Login:** `POST /api/auth/login` – returns JWT token
- 4 **Create course:** `POST /api/courses` with Bearer token – Atlas creates document with `_id` and `user`
- 5 **Get all:** `GET /api/courses` – returns real Atlas documents
- 6 **Update:** `PUT /api/courses/{_id}` with Bearer token
- 7 **Forbidden test:** `PUT` with a different user's token – expect 403
- 8 **Delete:** `DELETE /api/courses/{_id}` – document removed from Atlas

POST /api/courses – 201

```
{
  "_id": "64a3f8...",
  "title": "Cloud Computing",
  "dept": "CS",
  "credits": 3,
  "user": "64u7b2...",
  "createdAt": "2025-01-...",
  "--v": 0
}
```

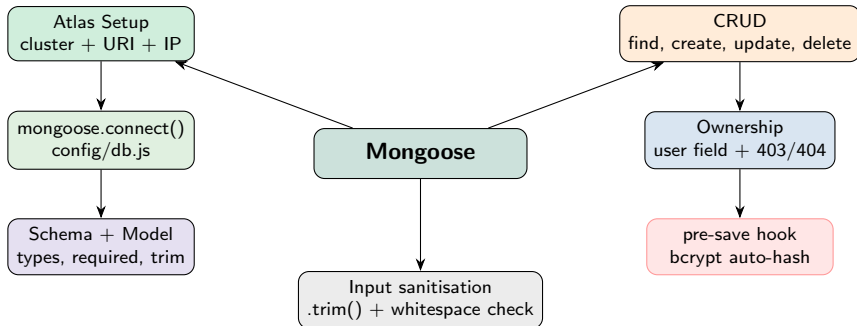
💡 Check Atlas in browser

After running Postman requests, open MongoDB Atlas in your browser, click on **Browse Collections** to see your data live – just like a database browser.

Outline

- 1 MongoDB Atlas Setup
- 2 `mongoose.connect()`
- 3 Schema: types, required, default, trim
- 4 Mongoose CRUD Operations
- 5 User Model with bcrypt pre-save hook
- 6 Ownership, Sanitisation & Wiring
- 7 Recap

MongoDB & Mongoose – Concept Map



Lecture 14 – Key Takeaways

- 1 **MongoDB Atlas** is a free cloud database; set up: create cluster → add user → whitelist IP → copy URI to `.env`
- 2 `mongoose.connect(MONGO_URI)` in `config/db.js`; call it before `app.listen()`; exit process on failure
- 3 A **Schema** defines field types, required, default, trim, unique; use full object syntax for validation
- 4 `mongoose.model('Course', schema)` creates the **Model**; collection name = plural lowercase (`courses`)
- 5 CRUD: `find()` for all, `findById()` for one, `create()` for insert, `findByIdAndUpdate({new:true})` for update, `findByIdAndDelete()` for delete
- 6 Add a user **field** (ObjectId ref) on create; compare `doc.user.toString()` with `req.user.id` before update/delete
- 7 Return **403 Forbidden** when the user exists but lacks permission; **404 Not Found** when the document doesn't exist
- 8 The **pre-save hook** hashes passwords automatically – check `this.isModified('password')` to avoid double-hashing
- 9 Always **sanitise input**: `.trim()` all strings, reject whitespace-only values, normalise before saving

Congratulations! You have built a full MERN-stack REST API with authentication and a real database.

Practice Exercises

- 1 Create a free MongoDB Atlas account. Build a cluster, add a database user, whitelist 0.0.0.0/0, and copy the connection string into `.env` as `MONGO_URI`. Run `npm run dev` and confirm MongoDB Atlas connected appears.
- 2 Define a **Course schema** with `title` (String, required, trim), `dept` (String, required, uppercase), `credits` (Number, default 3, min 1, max 6), and `timestamps: true`. Export the model as `Course`.
- 3 Replace the in-memory `courses` array in your routes with Mongoose calls. Test all 5 CRUD endpoints in Postman. Check that data persists after restarting the server.
- 4 Add a **pre-save hook** to the User model to auto-hash passwords with `bcrypt`. Register two users via Postman and check the Atlas users collection – confirm passwords are stored as hashes.
- 5 Add a user **field** (ObjectId ref) to the Course schema. Populate it on `Course.create()`. Then add ownership checks to the PUT and DELETE routes – test that a 403 is returned when User A tries to delete User B's course.
- 6 Test all **input sanitisation** cases in Postman: send a body with an empty string, a whitespace-only string, and a missing required field. Confirm 400 is returned for each.
- 7 **Challenge:** Add a GET `/api/courses?dept=SWE` query-string filter that uses `Course.find({dept})`. Add `sort` and `limit` options. Use Atlas “Browse Collections” to confirm only the matching documents are returned.